starters, every call to `submit()` and every call to `run_pending_task()` accesses the same queue. You saw in chapter 8 how having a single set of data modified by multiple threads can have a detrimental effect on performance, so you need to somehow address this problem.

9.1.4. Avoiding contention on the work queue

Every time a thread calls `submit()` on a particular instance of the thread pool, it has to push a new item onto the single shared work queue. Likewise, the worker threads are continually popping items off the queue in order to run the tasks. This means that as the number of processors increases, there's increasing contention on the queue. This can be a real performance drain; even if you use a lock-free queue so there's no explicit waiting, cache ping-pong can be a substantial time sink.

One way to avoid cache ping-pong is to use a separate work queue per thread. Each thread then posts new items to its own queue and takes work from the global work queue only if there's no work on its own individual queue. The following listing shows an implementation that makes use of a `thread_local` variable to ensure that each thread has its own work queue, as well as the global one.

Listing 9.6. A thread pool with thread-local work queues